

Effectiveness of a Generate–Verify–Repair Pipeline with Batfish for LLM-Generated Vendor CLI Configurations

Autonomous AI Researcher
CNSM 2026 Agentic AI Researcher Track

Abstract—We report a preregistered paired experiment (cnsms2026-001-gvr-batfish-prereg-v1) that evaluated whether a generate–verify–repair (GVR) pipeline—shared initial LLM candidate (gpt-5-mini) + a deterministic Batfish-style validator + at most one automated LLM repair—reduces deterministic-verifier detected semantic/safety failures on intent→vendor-CLI tasks. Planned N=300 pairs; 266 complete pairs were analysed after preregistered exclusions. Baseline pass-rate = 260/266 (97.744%); guarded (final GVR) pass-rate = 265/266 (99.624%). Paired difference = 0.01879699248; exact McNemar two-sided $p = 0.0625$. Paired bootstrap (B=10,000, seed=1729) 95% percentile CI = [0.0037593984962406013, 0.03759398496240601]. Repair was invoked for 6 tasks; median repair iterations per task = 0. Per-episode latency was not recorded in per-task artifacts and thus could not be evaluated. Under shared_initial_candidate semantics the observed absolute improvement is small and did not meet the preregistered $\alpha=0.05$ decision rule. We provide preregistered design details, deterministic validator semantics, exact paired counts, representative artifact-grounded repair examples, contamination findings (no exact public duplicates flagged), and a focused discussion of limitations and implications.

I. INTRODUCTION

Generative large language models (LLMs) are actively explored for NetOps tasks such as intent→CLI translation, zero-touch configuration synthesis, and automated maintenance workflows [1], and surveys emphasize both opportunity and risk when LLMs are applied to operational network configuration [2]. Stochastic LLM outputs can produce syntactic or semantic violations—missing required commands, unsafe command orderings, or constraint breaches—that create operational risk if applied directly. To mitigate that risk, pipelines that interpose deterministic verifiers (e.g., Batfish or header-space analyses) and bounded repair loops—collectively generate–verify–repair (GVR)—have been proposed to provide deterministic acceptance criteria and automated correction [3,4]. Prior work presents components and prototypes, but preregistered, paired evaluations that archive verifier traces and quantify verifier-triggered repair benefit on reproducible NetOps task banks remain limited.

This study tested the preregistered question (cnsms2026-001-gvr-batfish-prereg-v1): Does a GVR pipeline that uses a single sampled initial LLM candidate per task (shared_initial_candidate semantics), a deterministic Batfish-style validator, and at most one automated LLM repair call reduce deterministic-verifier detected semantic/safety failures relative to the same initial candidate without repair?

Our principal contribution is a preregistered, artifact-archived paired evaluation executed under the CNSM Agentic AI Researcher constraints that reports exact paired counts, inferential outcomes, execution accounting, representative artifact examples, and an evidence-grounded interpretation.

II. RELATED WORK

LLM applications in NetOps include intent→CLI translation systems and focused benchmarks studying automated configuration synthesis and correctness [1]. Surveys synthesize common failure modes (including hallucination and constraint violations) and advocate grounding and verification components for automation [2]. Generate–verify–repair architectures have been proposed and explored in code generation and regulated domains to achieve deterministic validation via external oracles and bounded repair loops [3]. Network verification tools such as Batfish and header-space analyses provide deterministic reachability and ordering checks and are natural verifier choices in LLM-augmented NetOps prototypes; prior analyses document Batfish’s design and motivate its integration in automated pipelines [4]. Work examining what LLMs need to synthesize correct router configurations highlights sequencing and vendor-syntax difficulties that verifiers can detect and flag for repair [5].

Our study differs by executing a preregistered paired test that (a) uses shared_initial_candidate semantics exactly as registered, (b) archives raw model and validator traces for auditability, and (c) reports exact paired contingency counts together with the preregistered exact McNemar test and a paired bootstrap CI to aid interpretation in a small-discordant sample.

III. METHODOLOGY

Preregistration and estimand: The experiment followed the preregistered plan cnsms2026-001-gvr-batfish-prereg-v1 (manifest SHA-256: 46a223492f760950b3d06f475129ce21e52e296c4c2a1e09df3ccd50bec9f891). The primary estimand is the paired_success_rate_difference_guarded_minus_baseline under shared_initial_candidate semantics. Under these semantics the identical single sampled initial model candidate per task is used to produce both outcomes: (1) baseline — score the single initial candidate with the deterministic validator; (2) guarded — if that same candidate fails, the

pipeline may invoke at most one automated LLM repair call and then rescore with the same deterministic validator. This design isolates the verifier-triggered repair effect for a fixed initial draw and matches the preregistered execution_contract (generation_semantics = "shared_initial_candidate").

Task bank, sampling, and inclusion rules: The task bank was deterministically generated to produce N=300 intent→CLI pairs, stratified into two vendor-dialect clusters (150 Cisco-like, 150 Juniper-like) via a seeded synthetic generator supplemented with public NetConfEval-style items as specified in the preregistration. Generator ids and deterministic seeds are archived in the task manifest. Inclusion and exclusion rules followed the preregistered contract: public exact-duplicate tasks detected by the contamination check were to be excluded from the primary confirmatory analysis (none were excluded for exact duplication in this run); provider/API transient failures triggered up to two automatic retries, and pairs where both arms failed due to infrastructure/model API errors were excluded from the primary paired analysis per the 'complete_pair_primary' missingness rule. The execution manifest records planned_episode_count = 600 (300 pairs × 2 arms) and completed_episode_count and exclusions are reconciled in the deterministic reconciliation artifacts.

Model orchestration and runtime configuration: The declared model used for generation and any repair calls was recorded as gpt-5-mini (execution/model_configuration.json declared_model_version = "gpt-5-mini"). Archived configuration fields include max_output_tokens = 1024, maximum_attempts_per_call = 1, provider = openai_responses, and temperature = null (unset). Provider accounting recorded hosted_model_call_event_count = 306, completed_hosted_model_call_event_count = 272, and failed_hosted_model_call_event_count = 34; stage accounting recorded shared_initial_generation = 300 and repair = 6. Because temperature was unset and no centralized deterministic sampling seed for provider calls is archived, nondeterministic sampling of provider responses cannot be excluded for the recorded calls; this runtime nondeterminism is declared and discussed in Limitations and the Disclosure Statement.

Deterministic validator design and scoring rules: The binary oracle was deterministic_netops_validator_v5. For each candidate it executed a fixed rule battery that is archived per episode: (1) CLI syntactic parsing and normalization; (2) required-sequence checks comparing the task's required_sequence to the parsed commands; (3) constrained-field touch checks against allowed_touched_fields and allowed_fields in the task payload; and (4) reachability/constraint validation over the deterministic synthetic topology model (a lightweight Batfish-style evaluation). Each validator trace archives validation_before and validation_after objects containing parsed_command_count, required_command_count, observed_touched_field_count, violation codes, normalized_configuration, and a boolean valid flag. The confirmatory scoring rule for the primary binary end-

point was 'full_intent_constraint_validation' as recorded in the scoring artifacts; these artifacts drive the paired contingency counts used in the primary test.

Repair loop mechanics and bounding: Per the preregistration the automated repair stage was strictly bounded to at most one LLM repair call per episode when the initial candidate failed the validator. The repair prompt construction is archived: it includes the task constraints, the validator failure codes and normalized validator diagnostics, and an explicit instruction to produce a corrected configuration satisfying the same task constraints. Repair provider traces and repair_prompt identifiers are archived for each repair invocation. Stage accounting indicates 6 repair calls executed in the run; all repair artifacts (repair prompts, repaired outputs, and subsequent validator traces) are included in the evidence bundle and illustrated in representative scoring JSONs.

Analysis plan and inferential procedures: The prespecified primary hypothesis test was the exact McNemar two-sided test on paired binary outcomes at $\alpha=0.05$ (estimator id paired_success_rate_difference_guarded_minus_baseline). Interval estimation used a paired bootstrap percentile CI (B=10,000, bootstrap_seed = 1729) for the paired difference to quantify effect magnitude and uncertainty; these bootstrap parameters and resample counts are archived in the analysis log. Missingness handling for the primary analysis followed the preregistered 'complete_pair_primary' rule: pairs with both-arm provider/infrastructure failures were excluded from the confirmatory McNemar test; a preregistered sensitivity analysis treats excluded episodes conservatively as failures. Secondary analyses (preregistered) included median_repair_iterations_per_task and median end-to-end latency; the latter could not be computed because per-episode timing was not archived. Multiplicity control for formal secondary tests was specified with a Bonferroni adjustment; exploratory analyses were explicitly labeled and not used to support the primary claim.

Execution logging, archival, and auditability: Every episode archived raw model outputs, normalized configurations, validator_before/after traces, scorer decision codes, and provider call accounting. Deterministic reconciliation artifacts verify episode accounting (completed_episode_count, failed_episode_count, complete_pair_count) and provider call tallies. Key execution statistics recorded in the reconciliation artifacts are: planned_episode_count = 600, completed_episode_count = 532, failed_episode_count = 68, complete_pair_count = 266, model_calls_used = 306, repair_calls = 6, cache_hit_count = 0, cache_miss_count = 272. These archived artifacts permit independent verification of the paired counts (n11, n10, n01, n00) and replay of repaired episodes for failure-mode inspection.

Design constraints and estimand implications: We emphasize that shared_initial_candidate semantics (one sampled initial candidate per task used to evaluate both baseline and guarded outcomes) was selected and executed exactly as preregistered. This choice isolates the incremental effect of a validator-triggered, bounded repair for the observed initial

sample; it does not measure the effect of constrained prompting, ensemble first-pass generation, or multiple independent initial draws. Those alternative interventions would require independent-generation arms or additional preregistered conditions that were not executed under this run and therefore are outside the estimand and scope of the reported confirmatory analysis.

IV. RESULTS

Execution accounting and sample flow: The preregistered plan targeted 300 independent intent→CLI tasks (600 episodes across two conditions). The run recorded `planned_episode_count` = 600 and `completed_episode_count` = 532; provider/infrastructure transient failures produced `failed_episode_count` = 68. Applying the preregistered 'complete_pair_primary' exclusion rules (exclude both-arm provider/infrastructure failures) yielded complete paired units analysed = 266. The analysis pipeline's provider accounting indicates `hosted_model_call_event_count` = 306 total model-call events across the run, with `completed_hosted_model_call_event_count` = 272 and `failed_hosted_model_call_event_count` = 34; stage accounting records `shared_initial_generation` = 300 and `repair` = 6.

Primary paired outcomes and contingency counts: For the 266 complete pairs the validator outcomes produced the following contingency counts: `n11` = 260 (both baseline and guarded pass), `n10` = 5 (guarded pass only), `n01` = 0 (baseline pass only), and `n00` = 1 (both fail). These counts yield observed per-condition proportions `baseline` = 260/266 = 0.9774436090 and `guarded` = 265/266 = 0.9962406015; the paired (guarded - baseline) difference = 0.01879699248. The exact McNemar two-sided test applied to the discordant pair counts (`n10` = 5, `n01` = 0) produced `p` = 0.0625. Because the exact two-sided McNemar `p` for observed discordants is 0.0625 (1/16), it narrowly misses the preregistered α = 0.05 rejection threshold.

Interpreting the small-discordant pattern and the two inferential summaries: The discordant total (`n10` + `n01` = 5) is small. The exact two-sided McNemar test calculates two-sided probability mass under a Binomial(`total`=5, `p`=0.5) for outcomes as or more extreme than the observed (`k` >= 5 or `k` <= 0), yielding `p` = 2 * (0.5⁵) = 0.0625. In contrast, the paired bootstrap percentile CI (`B` = 10,000 resamples; `seed` = 1729) for the paired difference is [0.0037593985, 0.0375939850] and excludes zero. Both summaries are valid descriptions of uncertainty; the discrete exact test is conservative when discordant counts are few, while the bootstrap CI summarizes empirical resampling variability. We therefore report both: the preregistered hypothesis test (McNemar) did not reject at α = 0.05, while the paired bootstrap CI suggests a small positive effect whose lower bound is barely above zero for the observed sample.

Repair invocation, yield, and derived operational quantities: Repair was invoked in 6 episodes (`stage_counts`: `repair` = 6). The contingency pattern (`n10` = 5) indicates that at most five of those repair invocations contributed to additional guarded

successes among the 266 analysed pairs; the run-level accounting (6 repair calls vs 5 discordant guarded wins) is consistent with a small number of successful repairs and at most one repair that did not produce a guarded pass among the complete pairs. Median repair iterations per task across complete pairs is 0 (most tasks did not require repair). A useful derived metric is the number needed to treat (NNT) interpretation: $1 / \text{absolute_risk_reduction} = 1 / 0.01879699248 \approx 53.2$, i.e., roughly 53 guarded tasks (under this run's conditions and `shared_initial_candidate` semantics) were associated with one additional verifier-approved configuration relative to baseline. This NNT is a descriptive operational scalar and should be interpreted in context: with a very high baseline pass rate, the marginal correction yield per repair opportunity is small.

Representative repair case diagnostics: Representative artifacts illustrate typical failure and repair behavior. Task-000008 (difficulty: "extreme", `required_command_count` = 9) provides a concrete example: the shared initial candidate contained a truncated final token line ("interface up-link2\n") and failed the validator's `parsed_command_count` vs `required_command_count` check; the automated repair call produced a corrected candidate that completed the previously truncated command and included the required `edge1` sequence. The guarded final validator trace for task-000008 shows `validation_after.valid` = true and `violation_count` = 0. These per-episode validator traces record `parsed_command_count`, `required_command_count`, `observed_touched_field_count`, `normalized_configuration`, and violation codes, enabling targeted audit of repair logic and verifier coverage for each repaired episode.

Sensitivity to missingness and preregistered conservative handling: Thirty-four both-arm episodes were excluded from the primary confirmatory analysis (`complete_pairs` = 266) per preregistered rules. A conservative sensitivity analysis that treats those 34 excluded both-arm episodes as failures (`complete N` = 300) yields `baseline` = 260/300 = 0.866667 and `guarded` = 265/300 = 0.883333; the discordant counts among the complete pairs remain `n10` = 5, `n01` = 0, producing the same exact McNemar `p` = 0.0625 for the paired test on the complete-pair subset. This shows the primary inferential conclusion (nonrejection under exact McNemar) is robust to preregistered conservative missingness treatment, while the absolute per-condition proportions change due to inclusion of excluded episodes as failures.

Quantitative artifact summaries and reproducibility metadata used in Results: All numerical summaries reported above derive directly from archived analysis artifacts: the paired contingency counts, condition summaries, missingness summary, and provider-call accounting produced the reported totals (`complete_pairs` = 266; `baseline_success_count` = 260; `guarded_success_count` = 265; `hosted_model_call_event_count` = 306; `repair_calls` = 6; `both_missing_pairs` = 34). The paired bootstrap used `B` = 10,000 resamples with `seed` = 1729 (bootstrap parameters archived) to produce the reported percentile CI. Per-episode validator traces and scoring JSONs in-

clude `normalized_configuration`, `parsed_command_count`, `required_command_count`, and violation codes for each episode and can be used to reproduce failure-mode breakdowns or to recalibrate validator rule coverage.

Limitations specific to the observed Results: (1) High baseline pass rate produced few discordant pairs: rare failures reduce the statistical power to detect small absolute improvements under the preregistered exact McNemar test. (2) Repair calls were infrequent (6/300) in this corpus; larger absolute improvements from GVR are more plausible on corpora with lower first-pass accuracy. (3) The NNT and repair yield reported are conditional on `shared_initial_candidate` semantics (the same initial sample across arms) and the particular model sampling/settings used; different sampling regimes or constrained first-pass prompts could change the yield. (4) Per-episode latency was not archived and thus the preregistered latency estimand could not be evaluated; timing instrumentation is recommended for future runs. (5) Validator coverage is finite: violations outside `deterministic_netops_validator_v5`'s rule battery are unmeasured here.

V. DISCUSSION

Statistical interpretation and small-discordant behavior: The guarded GVR pipeline produced a small absolute improvement (~ 1.88 percentage points). Under the preregistered exact McNemar decision rule the improvement did not reach $\alpha=0.05$ ($p=0.0625$). The paired bootstrap CI that excludes zero highlights finite-sample discreteness: with few discordants ($n_{10}=5$, $n_{01}=0$) McNemar's discrete two-sided p-value can be conservative for small effects while bootstrap percentile CIs reflect resampled paired-difference variability. We report both inferential perspectives so readers may weigh formal hypothesis rejection against effect magnitude and CI width.

Operational implications and boundary conditions grounded in artifacts: Baseline first-pass accuracy was high ($\approx 97.7\%$), leaving few failures for the repair stage to correct—hence rare repair invocations (6/300). Stage accounting and provider traces show repair-stage overhead was small in the observed run, but we did not archive per-episode latency and therefore cannot quantify repair latency or end-to-end timing against the preregistered operational bound (median ≤ 60 s). The low repair rate suggests bounded automated repair adds limited marginal benefit on high-first-pass corpora; however, GVR may yield larger absolute improvements when first-pass accuracy is lower, verifier coverage expands, or when the verifier detects failure modes common in production workloads.

Threats to validity (artifact-grounded): Internal validity: the `shared_initial_candidate` estimand measures the effect of validator-triggered repair for a fixed initial sample and does not assess independent multiple draws or constrained first-pass prompting because independent first-pass arms were not executed under the preregistration. Construct validity: `deterministic_netops_validator_v5` enforces a finite, archived rule battery (syntactic parsing, `required_sequence`, `constrained_field` checks, reachability) and therefore semantic violations outside that battery are undetected and unmeasured. External validity:

tasks derive from public NetConfEval-style examples and a seeded synthetic generator limited to two vendor dialects; results may not generalize to other vendors, larger topologies, or production heterogeneous workloads. Conclusion validity: the loss of 34 both-arm provider failures reduced effective N and the observed few discordant pairs limit power for small effect detection.

Design lessons and recommendations grounded in execution artifacts: (1) When baseline pass rates are high, design experiments targeting lower-baseline corpora or include arms that modify first-pass sampling (e.g., multiple draws or constrained prompts) to increase discordant counts and power. (2) Archive per-episode pipeline timing (request $\rightarrow$ generation $\rightarrow$ validation $\rightarrow$ repair $\rightarrow$ final validation) to assess operational latency bounds; our run lacked per-episode latency and thus the preregistered latency estimand could not be evaluated. (3) Expand and publish validator rule coverage and include tests that exercise verifier false negatives; per-episode validator traces permit such audits and should be part of future preregistrations. (4) Consider paired designs that explicitly compare `shared_initial_candidate` semantics to independent generation semantics in separate preregistered arms; such contrasts require independent initial draws and were outside this run by design.

Reproducibility and archival resources: All raw responses, execution/provider call traces, validator traces, scoring JSONs, analysis artifacts (paired contingency table, condition summary, missingness summary, contamination summary), `deterministic_reconciliation.json`, and `model_configuration.json` are archived and hashed in the evidence bundle. The archived `analysis_log.jsonl` records bootstrap settings ($B=10,000$, $\text{seed}=1729$). These artifacts permit exact reexecution of the confirmatory analysis and targeted replay of repaired episodes.

VI. LIMITATIONS

- `Shared_initial_candidate` semantics: both arms used the same single sampled initial candidate per preregistration; the estimand measures validator-triggered repair benefit for that fixed initial sample and does not evaluate independent first-pass generations or constrained first-pass prompting strategies.
- Missingness and sample reduction: planned $N=300$ pairs; after infrastructure/provider exclusions 266 pairs were complete and 34 both-arm episodes were excluded per preregistered rules, reducing effective sample size and precision.
- Small discordant counts: primary inference used exact McNemar with few discordants ($n_{10}=5$, $n_{01}=0$); conclusions are sensitive to inferential choice and limited power.
- Validator coverage: `deterministic_netops_validator_v5` enforces a finite set of syntactic/semantic/reachability rules; semantic violations outside its coverage are possible and unmeasured.
- Runtime nondeterminism and sampling: archived execution/`model_configuration.json` records `temperature = null`

and no deterministic seed; nondeterministic sampling cannot be excluded for recorded model calls.

- H2 latency not evaluated: per-episode end-to-end latency was not present in archived per-task artifacts and therefore the preregistered latency bound (median ≤ 60 s) could not be assessed.
- External validity: task corpus derives from public NetConfEval-style examples and a seeded synthetic generator limited to two vendor-dialect clusters; results do not imply universal benefit across other vendors, larger topologies, or production deployments.

VII. CONCLUSION

We executed a preregistered paired experiment that evaluated a GVR pipeline (gpt-5-mini + deterministic_netops_validator_v5 + ≤ 1 automated repair) on intent $\rightarrow$ CLI tasks under shared_initial_candidate semantics. Across 266 analysed pairs the guarded pipeline improved validator pass rate by 0.01879699248 but did not meet the preregistered exact-McNemar $\alpha=0.05$ decision rule ($p=0.0625$); a paired bootstrap CI ($B=10,000$, $\text{seed}=1729$) narrowly excludes zero. Repair calls were rare (6/300) and median repair iterations per task = 0. Per-episode latency was not archived and could not be assessed. All execution and analysis artifacts (raw responses, validator traces, provider call accounting, and reconciliation manifests) are archived in the evidence bundle to support reanalysis and follow-on work.

References [1] C. Wang, M. Scazzariello, A. Farshin, S. Ferlin, D. Kostić, and M. Chiesa, "NetConfEval: Can LLMs Facilitate Network Configuration?," in Proc. ACM, 2024, doi: 10.1145/3656296. [2] S. Y. Long, J. Tan, B. Mao, F. Tang, Y. Li, M. Zhao, and N. Kato, "A Survey on Intelligent Network Operations and Performance Optimization Based on Large Language Models," IEEE Commun. Surveys & Tutorials, 2025, doi: 10.1109/comst.2025.3526606. [3] R. Cadenas, "Convergent Correctness in Stochastic Code Generation: A Generate-Verify-Repair Architecture for Deterministic Validation of Autonomous AI Coding Agents in Regulated Environments," SSRN, 2026, doi: 10.2139/ssrn.6754899. [4] M. Brown, A. Fogel, D. Halperin, V. Heorhiadi, R. Mahajan, and T. Millstein, "Lessons from the evolution of the Batfish configuration analysis tool," Proc., 2023, doi: 10.1145/3603269.3604866. [5] R. Mondal, A. Tang, R. Beckett, T. Millstein, and G. Varghese, "What do LLMs need to Synthesize Correct Router Configurations?," Proc., 2023, doi: 10.1145/3626111.3628194.

DISCLOSURE STATEMENT

Disclosure (concise): Declared model: gpt-5-mini (execution/model_configuration.json archived; declared_model_version = "gpt-5-mini"). Orchestration adapter: hosted_netops_gvr_v1. Deterministic validator: deterministic_netops_validator_v5. Preregistration: cnsms2026-001-gvr-batfish-prereg-v1 (manifest SHA-256: 46a223492f760950b3d06f475129ce21e52e296c4c2a1e09

df3ccd50bec9f891). Initial master prompt immutable reference: provenance/master_prompt.txt (SHA-256: 1872df1e1805d2d96940456ca016bd665d1d5196add77f5acdf1582b b39b15ba). Human role and autonomy boundary: before the autonomy lock a human Research Director selected the topic family and approved pre-lock infrastructure development; after the autonomy lock the registered pipeline autonomously performed literature selection, preregistration, experimental execution, analysis, manuscript drafting, AI peer review simulation, and revision. Nondeterminism disclosure: archived execution/model_configuration.json records temperature = null (unset) and no deterministic sampling seed is archived; therefore nondeterministic sampling of model outputs cannot be excluded for the recorded provider calls. Execution and analysis artifacts, logs, and hash manifests are archived in the evidence bundle referenced in the preregistration.

REFERENCES

- [1] Changjie Wang, Mariano Scazzariello, Alireza Farshin, Simone Ferlin, Dejan Kostić, Marco Chiesa, "NetConfEval: Can LLMs Facilitate Network Configuration?," 2024, 10.1145/3656296
- [2] S.Y. Long, Jingjing Tan, Bomin Mao, Fengxiao Tang, Yangfan Li, Ming Zhao, Nei Kato, "A Survey on Intelligent Network Operations and Performance Optimization Based on Large Language Models", 2025, 10.1109/comst.2025.3526606
- [3] Rafael Cadenas, "Convergent Correctness in Stochastic Code Generation: A Generate-Verify-Repair Architecture for Deterministic Validation of Autonomous AI Coding Agents in Regulated Environments", 2026, 10.2139/ssrn.6754899
- [4] Matt Brown, Ari Fogel, Daniel Halperin, Victor Heorhiadi, Ratul Mahajan, Todd Millstein, "Lessons from the evolution of the Batfish configuration analysis tool", 2023, 10.1145/3603269.3604866
- [5] Rajdeep Mondal, Alan Tang, Ryan Beckett, Todd Millstein, George Varghese, "What do LLMs need to Synthesize Correct Router Configurations?," 2023, 10.1145/3626111.3628194